4

Advanced MVVM Concepts

After learning the basics of the MVVM pattern and its implementation in WinUI, it's now time to build on that knowledge base to handle some more advanced techniques. Now, you will learn how to keep components loosely coupled and testable when adding new dependencies to the project.

Few modern applications have only a single page or window. There are MVVM techniques that can be leveraged to navigate between pages from a **`ViewModel`** command without being coupled to the UI layer.

In this chapter, you will learn about the following concepts:

- Understanding the basics of **Dependency Injection** (**DI**)
- Leveraging DI to expose **`ViewModel`** classes to WinUI views
- Using MVVM and **`x:Bind`** to handle additional UI events with event handlers in the ViewModel
- Navigating between pages with MVVM and DI

By the end of this chapter, you will have a deeper understanding of the MVVM pattern and will know how to decouple your view models from any external dependencies.

Technical requirements

To follow along with the examples in this chapter, please reference the *Technical requirements* section in ***Chapter 2***, *Configuring the Development Environment and Creating the Project*.

You will find the code files for this chapter here: **https://github.com/PacktPublishing/Learn-WinUI-3-Second-Edition/tree/main/Chapter04**.

Understanding the basics of DI

Before starting down the path of using DI in our project, we should take some time to understand what DI is and why it is fundamental for building modern applications. You will often see DI referenced with another related concept, **Inversion of Control (IoC)**. Let's discuss these two concepts by doing the following:

- Clarify the relationship between them
- Prepare you to use DI properly in this chapter

DI is used by modern developers to inject dependent objects into a class rather than creating instances of the objects inside the class. There are several ways to inject those objects:

- **Method injection**: Objects are passed as parameters to a method in the class
- **Property injection**: Objects are set through properties
- **Constructor injection**: Objects are passed as constructor parameters

The most common method of DI is constructor injection. In this chapter, we will be using both property injection and constructor injection. Method injection will not be used because it is not common to use methods to set a single object's value in .NET projects. Most developers use properties for this purpose.

IoC is the concept that a class should not be responsible for (or have knowledge of) the creation of its dependencies. You're inverting control over object creation. This sounds a bit like DI, doesn't it? Well, DI is one method of achieving this IoC in your code. There are other ways to implement IoC, including the following:

- **Delegate**: This holds a reference to a method that can be used to create and return an object
- **Event**: Like a delegate, this is typically used in association with user input or other outside actions
- **Service Locator Pattern**: This is used to inject the implementation of a service at runtime

When you separate the responsibilities of object creation and use, it facilitates code reuse and increases testability.

The classes that will be taking advantage of DI in this chapter are views and ViewModels. So, if we will not be creating instances of objects in those classes, where will they be created? Aren't we just moving the tight coupling somewhere else? In a way, that is true, but the coupling will be minimized by centralizing it to one part of the project, the `App.xaml.cs` file. If you remember from the previous chapter, the `App` class is where we handle application-wide actions and data.

We are going to use a **DI container** in the `App` class to manage the application's dependencies. A DI container is responsible for creating and maintaining the lifetime of the objects it manages. The object's lifetime in the container is usually either *per instance* (each object request returns a new instance of the object) or a *singleton* (every object request returns the same instance of the object). The container is configured in the `App` class, and it makes instances available to other classes in the application.

In .NET 6 and later, DI is now a part of .NET itself. We will leverage the **host builder** configuration in .NET to register our application's depen-

dencies and resolve them in the classes where they are needed.

There are a number of other DI implementations that can be leveraged from MVVM frameworks. If you would like to explore some of them, here are their respective links:

- **Unity**: This DI implementation supports all types of .NET applications and has a full-featured IOC container (**http://unitycontainer.org/articles/introduction.html)**
- **DryIoc**: This small, lightweight IOC container supports .NET Standard 2.0 and .NET 4.5 and later applications (**https://github.com/dadhi/DryIoc**)
- **Prism**: This MVVM framework does not support WinUI 3, but developers can still leverage the DI capabilities (**https://prismlibrary.com/docs/dependency-injection/index.html**)

These concepts will be easier to understand as we implement the code in our application. Now, it's time to see DI and DI containers in practice.

Using DI with ViewModel classes

Most of the popular MVVM frameworks today include a DI container to manage dependencies. Because .NET now includes its own DI container, we will use that one. The .NET team has incorporated the DI container that used to be bundled with **ASP.NET Core**. It's both lightweight and easy to use. Luckily, this container is now available to all types of .NET projects via a **NuGet** package:

1. Open the project from the previous chapter or use the project in the `Start` folder in the GitHub repository for this chapter. In the `MyMediaCollection` project, open **NuGet Package Manager** and search for `Microsoft.Extensions.Hosting`:

Figure 4.1 – Microsoft's DI NuGet package

2. Select the package and install the latest stable version. After the installation completes, close the **NuGet Package Manager** tab and open **`App.xaml.cs`**. We will make a few changes here to start using the DI container.
   The DI container implements DI through interfaces called **`IHostBuilder`** and **`IServiceCollection`**. As the names imply, they are intended to create a collection of services for the application through a shared host. However, we can add any type of class to the container. Its use is not restricted to services.
   `IServiceCollection` builds the container, implementing the **`IServiceProvider`** interface. In the following steps, you will add support for DI to the application.
3. The first thing you should do is add a **`public`** property to the **`App`** class that makes the host container available to the project:

```
public static IHost HostContainer { get; private set;
}
```

   Here, **`get`** is public, but the property has a **`private set`** accessor. This restricts the creation of the container to the **`App`** class. Don't forget to add the required **`using`** statements to the code:

```
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
```

4. The next step is to create a new method that initializes the container, sets it to the **`public`** property, and adds our first dependency:

```
private void RegisterComponents()
{
    HostContainer = Host.CreateDefaultBuilder()
        .ConfigureServices(services =>
```

```
    {
        services.AddTransient<MainViewModel>();
    }).Build();
}
```

In the new **`RegisterComponents`** method, we are creating **`HostContainer`** and its service collection, registering **`MainViewModel`** as a **transient** (one instance per container request) object, and using the **`Build`** method to create and return the DI container. Although it's not strictly required, when adding multiple types to the container, it's a good practice to add dependent objects to the service collection first. We'll be adding more items to the container soon.

5. Finally, you will call **`RegisterComponents`** before creating the instance of **`MainWindow`** in the **`App.OnLaunched`** event handler:

```
protected override void
OnLaunched(LaunchActivatedEventArgs args)
{
    RegisterComponents();
    m_window = new MainWindow();
    m_window.Activate();
}
```

That's all the code needed to create and expose the DI container to the application. Now that we are delegating the creation of **`MainViewModel`** to the container, you can remove the property that exposes a static instance of **`MainViewModel`** from the **`App`** class.

Using the ViewModel controlled by the container is simple. Go ahead and open **`MainWindow.xaml.cs`** and update the **`ViewModel`** property to remove the initialization. Then, set the value of the **`ViewModel`** property using **`HostContainer.Services.GetService`** from the **`App`** class before the call to **`InitializeComponent`**:

```
public MainWindow()
{
    ViewModel = App.HostContainer.Services
      .GetService<MainViewModel>();
    this.InitializeComponent();
}
public MainViewModel ViewModel;
```

If you build and run the application now, it will work just as it did before. However, now our **`MainViewModel`** instance will be registered in the **`App`** class and managed by the container. As new models, view models, services, and other dependencies are added to the project, they can be added to the **`HostContainer`** in the **`RegisterComponents`** method.

We will be adding page navigation to the app later in this chapter. First, let's discuss the **event-to-command** pattern.

Leveraging x:Bind with events

In the previous chapter, we bound **`ViewModel`** commands to the **`Command`** properties of the **Add Item** and **Delete Item** buttons. This works great and keeps the ViewModel decoupled from the UI, but what happens if you need to handle an event that isn't exposed through a **`Command`** property? For this scenario, you have two options:

- Use a custom behavior such as **`EventToCommandBehavior`** in the .NET MAUI Community Toolkit. This allows you to wire up a command in the ViewModel to any event.
- Use **`x:Bind`** in the view to bind directly to an event handler on the view model.

In this application, we will use **`x:Bind`**. This option will provide compile-time type checking and added performance. If you want to learn

more about the .NET MAUI Community Toolkit, you can read the documentation on Microsoft Learn: **https://learn.microsoft.com/dotnet/communitytoolkit/maui/behaviors/event-to-command-behavior**.

We want to provide users of the **My Media Collection** application with the option to double-click (or double-tap) a row on the list to view or edit its details. The new **Item Details** window will be added in the next section. Until then, double-clicking an item will invoke the same code as the **Add Item** button, as this will become the **Add/Edit Item** button later:

1. Start by adding an **`ItemRowDoubleTapped`** event handler to the **`MainViewModel`** class that calls the existing **`AddEdit`** method:

```
public void ListViewDoubleTapped(object sender,
DoubleTappedRoutedEventArgs args)
{
    AddEdit();
}
```

2. Next, bind the **`ListView.DoubleTapped`** event to the ViewModel:

```
<ListView Grid.Row="1" ItemsSource="{x:Bind
    ViewModel.Items}"
    SelectedItem="{x:Bind
    ViewModel.SelectedMediaItem,
    Mode=TwoWay}"
    DoubleTapped="{x:Bind ViewModel
      .ListViewDoubleTapped}">
```

3. Finally, to ensure that the double-clicked row is also selected, modify the **`Grid`** inside **`ListView.ItemTemplate`** to set the **`IsHitTestVisible`** property to **`False`**:

```
<ListView.ItemTemplate>
    <DataTemplate x:DataType="model:MediaItem">
```

```
            <Grid IsHitTestVisible="False">
                ...
            </Grid>
        </DataTemplate>
    </ListView.ItemTemplate>
```

Now when you run the application, you can either click the **Add Item** button or double-click a row in the list to add new items. In the next section, you will update the **Add Item** button to be an **Add/Edit Item** button.

Page navigation with MVVM and DI

Until this point, the application has consisted of only a single window. Now it's time to implement page navigation by adding a host **`Frame`** and two **`Page`** objects so we can handle adding new items or editing existing items. The new **`Page`** will be accessible from the **Add/Edit Item** button or by double-clicking on an item in the list.

Migrating MainWindow to MainPage

If you're familiar with UWP app development, you should already understand page navigation. In UWP, the application consists of only a single window. At the root of the window, there is a **`Frame`** object, which hosts pages and handles the navigation between them. To achieve the same result in a desktop WinUI 3 app, we will create a new **`MainPage`**, move all the XAML content from **`MainWindow`** into **`MainPage`**, and update the **`App`** class to create a **`Frame`** as the new content of **`MainWindow`**. Then we can display the same contents by navigating to **`MainPage`**. Let's get started:

1. First, add a new folder to the project named **`Views`**.
2. Right-click the **Views** folder and select **Add** | **New Item**.

3. On the **Add New Item** dialog, select **WinUI** on the left and choose the **Blank Page (WinUI 3)** item template. Name the page **`MainPage`** and click **Create**.
4. Open **`MainWindow.xaml`** and cut the entire XAML contents of the **`Window`**.
5. Open **`MainPage.xaml`** and paste the XAML from **`MainWindow`**, replacing the empty **`Grid`** control.
6. You will also need to cut and paste the **`xmlns`** declaration for **`model`** from **`MainWindow`** to **`MainPage`**:

```
xmlns:model="using:MyMediaCollection.Model"
```

7. In **`MainWindow.xaml.cs`**, remove the **`ViewModel`** variable and the constructor code that fetches it from the **`HostContainer`**. Put this same code into **`MainPage.xaml.cs`**:

```
public MainPage()
{
    ViewModel = App.HostContainer.Services.GetService
      <MainViewModel>();
    this.InitializeComponent();
}
public MainViewModel ViewModel;
```

8. Next, open **`App.xaml.cs`** and add some code inside **`OnLaunched`** to create a **`rootFrame`**, add it to the **`MainWindow`**, and navigate to **`MainPage`** before activating the window:

```
protected override void OnLaunched
  (LauchActivatedEventArgs args)
{
    m_window = new MainWindow();
    var rootFrame = new Frame();
    RegisterComponents();
    rootFrame.NavigationFailed +=
      RootFrame_NavigationFailed;
    rootFrame.Navigate(typeof(MainPage), args);
```

```
        m_window.Content = rootFrame;
        m_window.Activate();
    }
    private void RootFrame_NavigationFailed(object sender,
      NavigationFailedEventArgs e)
    {
        throw new Exception($"Error loading page
          {e.SourcePageType.FullName}");
    }
```

We've also added an event handler to handle navigation failures for the **`Frame`**.

9. Make sure to add the necessary **`using`** statements to the file:

```
using System;
using Microsoft.UI.Xaml;
using Microsoft.UI.Xaml.Controls;
using Microsoft.UI.Xaml.Navigation;
using MyMediaCollection.Views;
```

If you run the app now, it should look and behave just as it did before, but now the controls are nested within a **`Page`** and a **`Frame`** on the **`Window`**. Let's add a second page and get ready to start navigating between our list and detail pages.

Adding ItemDetailsPage

The full **`ItemDetailsPage.xaml`** code can be found on GitHub (**https://github.com/PacktPublishing/Learn-WinUI-3-Second-Edition/blob/main/Chapter04/Complete/MyMediaCollection/Views/ItemDetailsPage.xaml**). You can follow along with the steps in this section or review the final code on GitHub.

NOTE

The project will not compile successfully until we have added the new ViewModel to the project and added it to the DI container for consump-

tion by the view. Before we add the ViewModel, we need to create some services to enable navigation and data persistence between views.

We will be showing **Item Details** in the same host window, and our content will reside within a new **`Page`** control. The **`Page`** will be set as the content and navigated to by the **`Frame`** we created. For more information about page navigation with WinUI, you can read this Microsoft Learn article: **https://learn.microsoft.com/windows/apps/design/basics/navigate-between-two-pages?tabs=wasdk**.

To add **`ItemDetailsPage`**, follow these steps:

1. Right-click the **Views** folder in **Solution Explorer** and select **Add** | **New Item**.
2. On the new item dialog, select **Blank Page (WinUI 3)** and name the page **`ItemDetailsPage`**.
3. There are going to be several input controls with some common attributes on the page. Start by adding three styles to a **`Page.Resources`** section just before the top-level **`Grid`** control:

```
<Page.Resources>
    <Style x:Key="AttributeTitleStyle"
      TargetType="TextBlock">
        <Setter Property="HorizontalAlignment"
          Value="Right"/>
        <Setter Property="VerticalAlignment"
          Value="Center"/>
    </Style>
    <Style x:Key="AttributeValueStyle"
       TargetType="TextBox">
        <Setter Property="HorizontalAlignment"
          Value="Stretch"/>
        <Setter Property="Margin" Value="8"/>
    </Style>
    <Style x:Key="AttributeComboxValueStyle"
       TargetType="ComboBox">
```

```
        <Setter Property="HorizontalAlignment"
          Value="Stretch"/>
        <Setter Property="Margin" Value="8"/>
    </Style>
</Page.Resources>
```

In the next step, we can assign **`AttributeTitleStyle`** to each **`TextBlock`**, **`AttributeValueStyle`** to each **`TextBox`**, and **`AttributeComboValueStyle`** to each **`ComboBox`**. If you need to add any other attributes to input labels later, you will only update **`AttributeTitleStyle`** and the attributes will automatically be applied to every **`TextBlock`** using that style.

4. The top-level **`Grid`** will contain three child **`Grid`** controls to partition the view into three areas—a header, the input controls, and the **Save** and **Cancel** buttons at the bottom. The input area will be given the bulk of the available space, so define **`Grid.RowDefinitions`** like this:

```
<Grid.RowDefinitions>
    <RowDefinition Height="Auto"/>
    <RowDefinition Height="*"/>
    <RowDefinition Height="Auto"/>
</Grid.RowDefinitions>
```

5. The header area will contain only a **`TextBlock`**. You are welcome to design this area however you like:

```
<TextBlock Text="Item Details" FontSize="18"
Margin="8"/>
```

6. The input area contains a **`Grid`** with four **`RowDefinitions`** and two **`ColumnDefinitions`** for the labels and input controls for the four fields that users can currently edit:

```
<Grid Grid.Row="1">
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto"/>
```

```
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="Auto"/>
    </Grid.RowDefinitions>
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="200"/>
        <ColumnDefinition Width="*"/>
    </Grid.ColumnDefinitions>
    <TextBlock Text="Name:" Style="{StaticResource
      AttributeTitleStyle}"/>
    <TextBox Grid.Column="1"
        Style="{StaticResource AttributeValueStyle}"
        Text="{x:Bind ViewModel.ItemName, Mode=TwoWay,
UpdateSourceTrigger=PropertyChanged}"/>
    <TextBlock Text="Media Type:" Grid.Row="1"
        Style="{StaticResource AttributeTitleStyle}"/>
    <ComboBox Grid.Row="1" Grid.Column="1"
        Style="{StaticResource AttributeCombox
          ValueStyle}"
        ItemsSource="{x:Bind ViewModel.ItemTypes}"
        SelectedValue="{x:Bind ViewModel
          .SelectedItemType, Mode=TwoWay}"/>
    <TextBlock Text="Medium:" Grid.Row="2"
        Style="{StaticResource AttributeTitleStyle}"/>
    <ComboBox Grid.Row="2" Grid.Column="1"
        Style="{StaticResource
          AttributeComboxValueStyle}"
        ItemsSource="{x:Bind ViewModel.Mediums}"
        SelectedValue="{x:Bind ViewModel
          .SelectedMedium, Mode=TwoWay}"/>
    <TextBlock Text="Location:" Grid.Row="3"
        Style="{StaticResource AttributeTitleStyle}"/>
    <ComboBox Grid.Row="3" Grid.Column="1"
        Style="{StaticResource
          AttributeComboxValueStyle}"
        ItemsSource="{x:Bind ViewModel.LocationTypes}"
        SelectedValue="{x:Bind ViewModel
          .SelectedLocation,Mode=TwoWay}"/>
</Grid>
```

7. The item's **`Name`** is a free-text entry field, while the others are **`ComboBox`** controls to allow the user to pick values from lists bound to **`ItemsSource`**. The final child element of the top-level **`Grid`** is a right-aligned horizontal **`StackPanel`** containing the **Save** and **Cancel** buttons:

```
<StackPanel Orientation="Horizontal"
        Grid.Row="2" HorizontalAlignment="Right">
    <Button Content="Save" Margin="8,8,0,8"
        Command="{x:Bind ViewModel.SaveCommand}"/>
    <Button Content="Cancel" Margin="8"
        Command="{x:Bind ViewModel.CancelCommand}"/>
</StackPanel>
```

The next stage is to add interfaces and services, so let's work on this next.

Adding new interfaces and services

Now that we have more than a single page to manage in the application, we need some services to centralize the page management and abstract the details from the **`ViewModel`** code. Start by creating **`Services`** and **`Interfaces`** folders in the project. Each service will implement an interface. This interface will be used for DI and later, if you were to add unit tests to a test project.

Creating a navigation service

The first service we need is a **navigation service**. Start by defining the **`INavigationService`** interface in the **`Interfaces`** folder. The interface defines methods to get the current page name, navigate to a specific page, or navigate back to the previous page:

```
public interface INavigationService
```

```
{
string CurrentPage { get; }
void NavigateTo(string page);
void NavigateTo(string page, object parameter);
void GoBack();
}
```

Now, create a **NavigationService** class in the **Services** folder. In the class definition, make sure that **NavigationService** implements the **INavigationService** interface. The full class can be viewed on GitHub (**https://github.com/PacktPublishing/Learn-WinUI-3-Second-Edition/blob/master/Chapter04/Complete/MyMediaCollection/Services/NavigationService.cs**). Let's discuss a few highlights.

The purpose of a navigation service in MVVM is to store a collection of available pages in the application so that when its **NavigateTo** method is called, the service can find a page that matches the requested **Name** or **Type** and navigate to it.

The collection of pages will be stored in a **ConcurrentDictionary<T>** collection. The **ConcurrentDictionary<T>** functions like the standard **Dictionary<T>**, but it can automatically add locks to prevent changes to the dictionary simultaneously across multiple threads:

```
private readonly IDictionary<string, Type> _pages = new
ConcurrentDictionary<string, Type>();
```

The **Configure** method will be called when you create **NavigationService** before adding it to the DI container. This method is not a part of the **INavigationService** interface and will not be available to classes that consume the service from the container. There is a check here to ensure views are only added to the service once. We check the dictionary to determine whether any pages of the same data type exist. If this condition is **true**, then the page has already been registered:

```
public void Configure(string page, Type type)
{
    if (_pages.Values.Any(v => v == type))
    {
        throw new ArgumentException($"The {type.Name} view
          has already been registered under another
            name.");
    }
    _pages[page] = type;
}
```

These are the implementations of the three navigation methods in the service. The two **`NavigateTo`** methods navigate to a specific page, with the second providing the ability to pass a parameter to the page. The third is **`GoBack`**, which does what you would think: it navigates to the previous page in the application. They wrap the **`Frame`** navigation calls to abstract the UI implementation from the view models that will be consuming this service:

```
public void NavigateTo(string page)
{
    NavigateTo(page, null);
}
public void NavigateTo(string page, object parameter)
{
    if (!_pages.ContainsKey(page))
    {
        throw new ArgumentException($"Unable to find a page
          registered with the name {page}.");
    }
    AppFrame.Navigate(_pages[page], parameter);
}
public void GoBack()
{
    if (AppFrame?.CanGoBack == true)
    {
        AppFrame.GoBack();
```

```
    }
}
```

We're ready to start using **`NavigationService`**, but first, let's create a data service for the application.

NOTE

You can jump ahead to implementing the services in the next section if you like. The **`DataService`** *and* **`IDataService`** *code is available in the completed solution on GitHub:* **https://github.com/PacktPublishing/Learn-WinUI-3-Second-Edition/tree/master/Chapter04/Complete/MyMediaCollection**.

Creating a data service

The data on **`MainPage`** of **My Media Collection** currently consists of a few sample records created and stored in **`MainViewModel`**. This isn't going to work very well across multiple pages. By using a data service, view models will not need to know how the data is created or stored.

For now, the data will still be sample records that are not saved between sessions. Later, we can update the data service to save and load data from a database without any changes to the view models that use the data.

The first step is to add an interface named **`IDataService`** to the **`Interfaces`** folder:

```
public interface IDataService
{
    IList<MediaItem> GetItems();
    MediaItem GetItem(int id);
    int AddItem(MediaItem item);
    void UpdateItem(MediaItem item);
    IList<ItemType> GetItemTypes();
```

```
    Medium GetMedium(string name);
    IList<Medium> GetMediums();
    IList<Medium> GetMediums(ItemType itemType);
    IList<LocationType> GetLocationTypes();
    int SelectedItemId { get; set; }
}
```

These methods should look familiar to you from previous chapters, but let's briefly review the purpose of each:

- **`GetItems`**: Returns all the available media items
- **`GetItem`**: Finds a media item with the provided **`id`**
- **`AddItem`**: Adds a new media item to the collection
- **`UpdateItem`**: Updates a media item in the collection
- **`GetItemTypes`**: Gets the list of media item types
- **`GetMedium`**: Gets a **`Medium`** with the provided name
- **`GetMediums`**: These two methods either get all available mediums or any available for the provided **`ItemType`**
- **`GetLocationTypes`**: Gets all the available media locations
- **`SelectedItemId`**: Persists the ID of the selected item on **`MainPage`**

Now, create the **`DataService`** class in the **`Services`** folder. Make sure that **`DataService`** implements **`IDataService`** in the class definition.

Again, we will only review parts of the code. You can review the entire implementation on GitHub (**https://github.com/PacktPublishing/Learn-WinUI-3-Second-Edition/blob/master/Chapter04/Complete/MyMediaCollection/Services/DataService.cs**). The data in **`DataService`** will be persisted in four lists and the **`SelectedItemId`** property:

```
private IList<MediaItem> _items;
private IList<ItemType> _itemTypes;
private IList<Medium> _mediums;
```

```
private IList<LocationType> _locationTypes;
public int SelectedItemId { get; set; }
```

Copy the **`PopulateItems`** method from **`MainViewModel`** and modify it to use **`List<T>`** collections and add the **`Location`** property assignment to each item.

Start by creating the three **`MediaItem`** objects:

```
var cd = new MediaItem
{
    Id = 1,
    Name = "Classical Favorites",
    MediaType = ItemType.Music,
    MediumInfo = _mediums.FirstOrDefault(m => m.Name ==
      "CD"),
    Location = LocationType.InCollection
};
var book = new MediaItem
{
    Id = 2,
    Name = "Classic Fairy Tales",
    MediaType = ItemType.Book,
    MediumInfo = _mediums.FirstOrDefault(m => m.Name ==
      "Hardcover"),
    Location = LocationType.InCollection
};
var bluRay = new MediaItem
{
    Id = 3,
    Name = "The Mummy",
    MediaType = ItemType.Video,
    MediumInfo = _mediums.FirstOrDefault(m => m.Name ==
      "Blu Ray"),
    Location = LocationType.InCollection
};
```

Then, initialize the **`_items`** list and add the three **`MediaItem`** objects you just created:

```
_items = new List<MediaItem>
{
    cd,
    book,
    bluRay
};
```

There are three other methods to pre-populate the sample data: **`PopulateMediums`**, **`Populate ItemTypes`**, and **`PopulateLocationTypes`**. All of these are called from the **`Data Service`** constructor. These methods will be updated later to use an **SQLite** data store for data persistence.

Most of the **`Get`** method implementations are very straightforward. The **`GetMediums(ItemType itemType)`** method uses **Language Integrated Query** (**LINQ**) to find all **`Medium`** objects for the selected **`ItemType`**:

```
public IList<Medium> GetMediums(ItemType itemType)
{
    return _mediums
        .Where(m => m.MediaType == itemType)
        .ToList();
}
```

NOTE

If you are not familiar with LINQ expressions, Microsoft has some good documentation on the topic: **https://learn.microsoft.com/dotnet/csharp/programming-guide/concepts/linq/**.

The **`AddItem`** and **`UpdateItems`** methods are also simple. They add to and update the **`_items`** collection:

```
public int AddItem(MediaItem item)
{
    item.Id = _items.Max(i => i.Id) + 1;
    _items.Add(item);
    return item.Id;
}
public void UpdateItem(MediaItem item)
{
    var idx = -1;
    var matchedItem = (from x in _items
                       let ind = idx++
                       where x.Id == item.Id
                       select ind).FirstOrDefault();
    if (idx == -1)
    {
        throw new Exception("Unable to update item. Item
          not found in collection.");
    }
    _items[idx] = item;
}
```

The **`AddItem`** method has some basic logic to find the highest **`Id`** and increment it by **`1`** to use at the new item's **`Id`**. **`Id`** is also returned to the calling method in case the caller needs the information.

The services are all created. It is time to set them up when the application launches and consume them in the view models.

Increasing maintainability by consuming services

Before using the services in view models, open the **`RegisterServices`** method in **`App.xaml.cs`** and add the following code to register the new services in the DI container and register a new **`ItemDetailsViewModel`** (yet to be created). We're also adding a parameter to the method to pass along to the constructor of the **`NavigationService`**. This will provide access to the **`Frame`** for page navigation:

```
private IServiceProvider RegisterServices(Frame rootFrame)
{
var navigationService = new NavigationService(rootFrame);
navigationService.Configure(nameof(MainPage),
typeof(MainPage));
navigationService.Configure(nameof(ItemDetailsPage),
typeof(ItemDetailsPage));
HostContainer = Host.CreateDefaultBuilder()
    .ConfigureServices(services =>
    {
        services.AddSingleton<INavigationService>
          (navigationService);
        services.AddSingleton<IDataService, DataService>();
        services.AddTransient<MainViewModel>();
        services.AddTransient<ItemDetailsViewModel>();
    }).Build();
}
```

Both **`INavigationService`** and **`IDataService`** are registered as **singletons**. This means that there will be only a single instance of each stored in the container. Any state held in these services is shared across all classes that consume them.

You will notice that when we're registering **`INavigationService`**, we are passing the instance we already created to the constructor. This is a feature of Microsoft's DI container and most other DI containers. It allows for initialization and configuration of instances before they're added to the container.

We need to make a few changes to **`MainViewModel`** to consume **`IDataService`** and **`INavigationService`**, update the **`PopulateData`** method, and navigate to **`ItemDetailsPage`** when **`AddEdit()`** is invoked:

1. Start by adding properties to **`MainViewModel`** for **`INavigationService`** and **`IDataService`**:

```
private INavigationService _navigationService;
private IDataService _dataService;
```

Don't forget to add a **`using`** statement for **`MyMediaCollection.Interfaces`**.

2. Next, update the constructor to receive and store the services:

```
public MainViewModel(INavigationService
navigationService, IDataService dataService)
{
    _navigationService = navigationService;
    _dataService = dataService;
    PopulateData();
}
```

Wait, we've added two parameters to the constructor but haven't changed the code that adds them to the DI container. How does that work? Well, the container is smart enough to pass them because both of those interfaces are also registered. Pretty cool!

3. Next, update **`PopulateData`** to get the data the view model needs from **`_dataService`**:

```
public void PopulateData()
{
    items.Clear();
    foreach(var item in _dataService.GetItems())
    {
        items.Add(item);
    }
    allItems = new
    ObservableCollection<MediaItem>(Items);
    mediums = new ObservableCollection<string>
    {
        AllMediums
    };
    foreach(var itemType in _dataService
      .GetItemTypes())
```

```
        {
            mediums.Add(itemType.ToString());
        }
        selectedMedium = Mediums[0];
    }
```

You need to add the **`AllMediums`** string constant with a value of **`"All"`** to the **`mediums`** collection because it's not part of the persisted data. It's only needed for the UI filter. Be sure to add this constant definition to **`MainViewModel`**.

4. Finally, when the hidden **`AddEditCommand`** calls the **`AddEdit`** method, instead of adding hardcoded items to the collection, you will pass **`selectedItemId`** as a parameter when navigating to **`ItemDetailsPage`**:

```
private void AddEdit()
{
    var selectedItemId = -1;
    if (SelectedMediaItem != null)
    {
        selectedItemId = SelectedMediaItem.Id;
    }
    _navigationService.NavigateTo("ItemDetailsPage",
      selectedItemId);
}
```

That's it for **`MainViewModel`**. Now let's work on the **`ItemDetailsPage`**.

Handling parameters in ItemDetailsPage

To accept a parameter passed from another page during navigation, you must override the **`OnNavigatedTo`** method in **`ItemDetailsPage.xaml.cs`**. The **`NavigationEventArgs`** parameter contains a property named **`Parameter`**. In our case, we passed an **`int`** containing the selected item's **`Id`**. Cast this **`Parameter`** property to **`int`** and

pass it to a method on the **`ViewModel`** named **`InitializeItemDetailData`**, which will be created in the next section:

```
protected override void OnNavigatedTo(NavigationEventArgs
e)
{
    base.OnNavigatedTo(e);
    var itemId = (int)e.Parameter;
    if (itemId > 0)
    {
        ViewModel.InitializeItemDetailData(itemId);
    }
}
```

In the next section, we'll add the final piece of the puzzle, the **`ItemDetailsViewModel`** class.

Creating the ItemDetailsViewModel class

To add or edit items in the application, you will need a view model to bind to **`ItemDetails Page`**. Right-click the **`ViewModels`** folder in **Solution Explorer** and add a new class named **`ItemDetailsViewModel`**.

The class will inherit from **`ObservableObject`** like **`MainViewModel`**. The full class can be found on GitHub at **https://github.com/PacktPublishing/Learn-WinUI-3-Second-Edition/blob/master/Chapter04/Complete/MyMediaCollection/ViewModels/ItemDetailsViewModel.cs**. Let's review some of the important members of the class.

The constructor receives the two services from the container and calls **`PopulateLists`** to populate **`ComboBox`** data from the data service:

```
public ItemDetailsViewModel(INavigationService
```

```
  navigationService, IDataService dataService)
{
    _navigationService = navigationService;
    _dataService = dataService;
    PopulateLists();
}
```

A **`public`** method named **`InitializeItemDetailData`** will accept the **`itemId`** parameter passed by **`ItemDetailsPage.OnNavigatedTo`**. It will call methods to populate the lists and initializes an **`IsDirty`** flag to enable or disable the **`SaveCommand`**:

```
public void InitializeItemDetailData(int itemId)
{
    _selectedItemId = itemId;
    PopulateExistingItem(_dataService);
    IsDirty = false;
}
```

The **`PopulateExistingItem`** method will add existing item data if the page is in edit mode, and **`PopulateLists`**, called from the constructor, fills the drop-down data to be bound to the view:

```
private void PopulateExistingItem(IDataService dataService)
{
    if (_selectedItemId > 0)
    {
        var item = _dataService.GetItem(_selectedItemId);
        Mediums.Clear();
        foreach (string medium in dataService.GetMediums
          (item.MediaType).Select(m => m.Name))
            Mediums.Add(medium);
        _itemId = item.Id;
        ItemName = item.Name;
        SelectedMedium = item.MediumInfo.Name;
        SelectedLocation = item.Location.ToString();
        SelectedItemType = item.MediaType.ToString();
```

```
    }
}
private void PopulateLists()
{
    ItemTypes.Clear();
    foreach (string iType in Enum.GetNames
      (typeof(ItemType)))
        ItemTypes.Add(iType);
    LocationTypes.Clear();
    foreach (string lType in Enum.GetNames
      (typeof(LocationType)))
        LocationTypes.Add(lType);
    Mediums = new TestObservableCollection<string>();
}
```

Most of this view model's properties are straightforward, but **`SelectedItemType`** has some logic to repopulate the list of **`Mediums`** based on the **`ItemType`** selected. For instance, if you are adding a book to the collection, there's no need to see the DVD or CD mediums in the selection list. We'll handle this custom logic in **`OnSelectedItemTypeChanged`**:

```
partial void OnSelectedItemTypeChanged(string value)
{
    IsDirty = true;
    Mediums.Clear();
    if (!string.IsNullOrWhiteSpace(value))
    {
        foreach (string med in _dataService.GetMediums
          ((ItemType)Enum.Parse(typeof(ItemType),
            SelectedItemType)).Select(m => m.Name))
            Mediums.Add(med);
    }
}
```

Lastly, let's look at the code that **`SaveCommand`** and **`CancelCommand`** will invoke to save and navigate back to **`MainPage`**:

```
private void Save()
{
    MediaItem item;
    if (_itemId > 0)
    {
        item = _dataService.GetItem(_itemId);
        item.Name = ItemName;
        item.Location = (LocationType)Enum.Parse
          (typeof(LocationType), SelectedLocation);
        item.MediaType = (ItemType)Enum.Parse(typeof
          (ItemType), SelectedItemType);
        item.MediumInfo = _dataService.GetMedium
          (SelectedMedium);
        _dataService.UpdateItem(item);
    }
    else
    {
        item = new MediaItem
        {
            Name = ItemName,
            Location = (LocationType)Enum.Parse
              (typeof(LocationType), SelectedLocation),
            MediaType = (ItemType)Enum.Parse(typeof
              (ItemType), SelectedItemType),
            MediumInfo = _dataService.GetMedium
              (SelectedMedium)
        };
        _dataService.AddItem(item);
    }
    _navigationService.GoBack();
}
private void Cancel()
{
    _navigationService.GoBack();
}
```

The other change needed before you run the application to test the new page is to consume **`ItemDetailsViewModel`** from **`ItemDetailsPage.xaml.cs`**:

```
public ItemDetailsPage()
{
    ViewModel = App.HostContainer.Services.GetService
      <ItemDetailsViewModel>();
    this.InitializeComponent();
}
public ItemDetailsViewModel ViewModel;
```

Now, run the app and try to add or edit an item—you should see the new page. If you are editing, you should also see the existing item data in the controls:

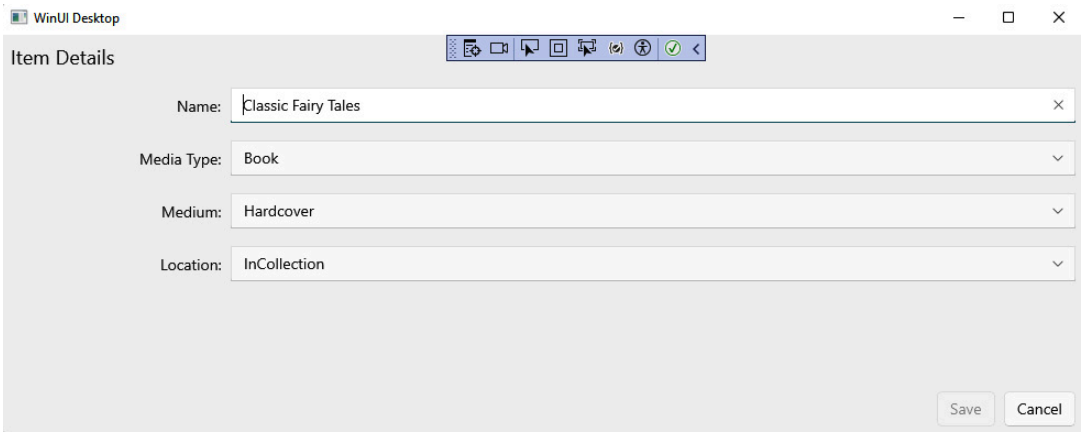

Figure 4.2 – The Item Details page with edited data populated

Great! Now when you save, you should see any added records or edited data appear on **`MainPage`**. Things are really starting to take shape in our project. Let's review what we have learned about WinUI and MVVM in this chapter.

Summary

You have learned quite a bit about MVVM and WinUI page navigation in this chapter. You have learned how to create and consume services in your application, and you have leveraged DI and DI containers to keep your view models and services loosely coupled. Understanding and using DI is key to building testable, maintainable code. At this

point, you should have enough knowledge to create a robust, testable WinUI application.

In the next chapter, you will learn about more of the available controls and libraries in WinUI 3.

Questions

1. How do DI and IoC relate?
2. How do you navigate to the previous page in a WinUI application?
3. What object do we use to manage dependencies?
4. With Microsoft's DI container, what method can you call to get an object instance?
5. What is the name of the framework that queries objects in memory?
6. What event argument property can you access to get a parameter passed to another **Page**?
7. Which dictionary type is safe to use across threads?